

Getting More From Your Models

A field guide to prompting and model strategy for someone who already knows his way around an LLM.



Start from where you already are

You are not a beginner, so this does not start at the beginning. You already do the two things most people never learn: you give the model real context, and you say what artifact you want. Your voice memos arrive carrying the why behind the ask, the audience, the format, even the personal detail that makes an example land. That is most of the game. This guide is about the smaller set of moves that separate a strong user from one who gets frontier results for less money: saying what done looks like, choosing how hard the model should think, and picking the model on purpose instead of sending everything to one.

The shift that happened under you

Prompting in 2026 is not the trick collection it was in 2022. The early advice (say think step by step, stack five examples, assign a persona) was scaffolding for models that could not yet reason on their own. The techniques that used to live in your prompt, writing out the reasoning steps by hand [2] and sampling several answers to vote on the best one [3], are now largely internal to the model. OpenAI says it plainly in its own reasoning guidance: telling these models to think step by step is unnecessary, and you should start with zero examples and add them only if the model fails [1]. The work moved to two other places: giving the model the right context, and being exact about the goal. Andrej Karpathy's name for the new job, context engineering, is the better frame. The craft is filling the window with exactly what the next step needs, and no more.

The three-tier mind

The second shift is economic. There is no longer one Claude, there are three, and they are priced far apart. The diagram above is the whole decision in one picture. Make Sonnet your default, drop to Haiku when the task is simple and you do it often, and climb to Opus only when you have evidence that Sonnet falls short. The question that saves the most money is not should I use Opus, it is could Haiku already do this.

Prompt the goal, not the procedure

The craft that is left once the model reasons on its own.

"You've hired a temp agency to send someone to do this task. This person arrives, they're pretty competent, they know a lot about your industry ... but they don't know the name of your company ... what would you say to that person?"

Amanda Askill, Anthropic prompt-engineering panel [V]

Say what done looks like

Your prompts name the task and the artifact. The one ingredient they almost always skip is the finish line. When you asked the team to teach you Wwise and Unity, you gave lovely context (you like animation and games, you are a beginner coder, you want six pages and an image of you at work) and then said, out loud, what would I make, I don't know. That is the missing piece, and naming it is the whole upgrade. A success criterion does not have to be precise, it has to exist. It gives the model a target to aim the entire response at.

BEFORE	Make a PDF that teaches me about Wwise and Unity. Use Perplexity. I am a beginner coder, so make it useful for a beginner. Six pages, and use Gemini for an image of me working on Wwise. What would I make? I don't know.
AFTER	Make a six-page beginner packet on Wwise and Unity. I like animation and games and I am a brand-new coder. I will know it worked if afterward I can say in one sentence what Wwise does, name one small project I could try this month, and recognize three terms a Unity audio tutorial would use. Use whatever sources and images get me there.

Same context, same scope. One sentence added (the finish line) and the tool names dropped so the system can pick the best path.

Show the model the task like you would show a person

The most useful picture from the people who prompt for a living is not technical. Picture that competent temp who just walked in. They know your field but not your students, your course, or what you actually want. What would you tell them? That is the prompt. It turns the literalness of the newest models from a quirk into a contract: Claude 4.x does what you actually wrote, so the fix for a wrong answer is rarely a cleverer phrasing. It is the sentence of context you forgot you knew.

Two debugging moves you are not using

When a prompt underperforms, the experts do two things most people skip. Before running it, they hand the model the instructions and say: do not do this yet, just tell me what is ambiguous. After a wrong answer, they say: you got this wrong, think about why, and rewrite my instructions so you would not. Often the model fixes your prompt for you. You already run a verifier on your PDFs. This is the same instinct pointed one step upstream, at the instructions themselves. And read the outputs, all of them, closely. The mistakes you never fix are the ones you never read.

Bang for your buck

Three levers most users never touch, because the cost is invisible.

Tier on purpose

The pricing, verified today: Haiku 4.5 is one dollar in and five out per million tokens, Sonnet 4.6 is three and fifteen, Opus is five and twenty-five [5]. The pattern the model makers recommend is a router: a cheap model triages and handles simple high-volume calls, a mid model does the bulk, and the top model is reserved for the slice that truly needs it. In your world it maps cleanly. The first pass over a voice memo (what is this, who does it mention, what type of task) is Haiku work. Drafting a worksheet or a lesson plan is Sonnet work. Architecting a new corner of KRAMOS, or a knotty debugging session, is where Opus earns its price.

Make thinking a dial

On the newest models, reasoning is a setting, not a sentence. You do not type think harder, you set an effort level (low to max) and turn on adaptive thinking, which lets the model decide how much to reason and interleave it between tool calls [4]. Two things worth knowing: on the latest Opus, thinking is off until you ask for it, and Sonnet's sweet spot for everyday work is medium effort, not the high it defaults to. Thinking tokens bill at full output rates even when you never see them, so an effort level left too high is a quiet tax.

Cache the things you repeat

Every memo your system processes ships the same large preamble: your CLAUDE.md, your SOPs, the routing prompt. That block is identical every time, which is exactly what prompt caching is for. A cache read costs a tenth of a normal input token and pays for itself after a single reuse [5]. Put the cache breakpoint at the end of the stable material and let the memo text float after it. For a system that runs all day on the same preamble, that is the difference between paying for it once an hour and paying for it on every call. If a job is not time-sensitive, the Batch API is half price on input and output, and the discount stacks on top of caching [5].

Lever	What it does	Reach for it
Right tier	Haiku to Opus is a 5x spread	Every task. Start Sonnet, justify Opus
Effort + adaptive	Reasoning depth as a dial, low to max	Lower it for simple work, thinking bills as output
Prompt caching	Cache reads cost 0.1x, breakeven after one reuse	Any stable prefix you resend (SOPs, CLAUDE.md)
Batch API	50% off in and out, stacks with caching	Anything that is not time-sensitive

One more watch-out: the newest Opus can use up to a third more tokens for the same text, so a migration that looks flat on price can still cost more per task [5].

Leveling up the system you built

You did not pick up a chatbot habit. You built an operating system. Here is where it goes next.

Give your verifier real criteria

Your orchestrator-and-specialists shape is exactly the architecture the model makers now recommend out loud: start with an orchestrator and subagents, the pattern that covers the widest range of work for the least overhead [6]. One sharp upgrade: a verifier told only to check whether something is good will rubber-stamp it [6]. Your QA gate works because it has a checklist. Carry that everywhere. When one agent checks another, hand it the explicit criteria, not just the word review.

Spin off subagents for clean context, not just speed

The underrated reason to delegate is not parallelism, it is context isolation: each subagent works in its own window and returns only the curated result, so the orchestrator stays lean [7]. This matters because of a measured limit. Every frontier model, Claude included, gets less accurate as its context fills, an effect now named context rot [8]. More context is not better context. When you do load a long prompt, put the instruction at the very start or the very end, because models read the edges of a long context far better than the middle [10]. Better still, load context just in time, a path or a link the model can follow, rather than stuffing everything in up front [11].

Build evals from your own misfires

You generate the perfect test set without trying. Every worksheet that comes back wrong, every memo routed to the wrong specialist, is an eval case. Collect twenty to fifty of these real failures and check changes against them, instead of trusting a feeling that this prompt is better now [9]. Track whether a fix ever works versus always works. Those are different bars, and which one you need depends on the task [9].

A weekly half hour

1. Read one full model output end to end, not just the answer. Note one thing it did that you did not ask for.
2. Take one ask you repeat and add a single success-criterion sentence to the template.
3. Push one routine call down to Haiku and one hard call up to Opus. See if you can tell.
4. Find one stable block you resend and put a cache breakpoint after it.
5. Save one prompt that worked and one that failed, each with a note on why. That file becomes your taste.

You are already in the top few percent here, not because you know secret prompts, but because you built a system and feed it well. The gains from here are not bigger prompts. They are a finish line on each ask, a thinking dial set on purpose, and the discipline to send the easy work to the cheap model. Keep going.

Sources and further reading

1. Reasoning best practices. OpenAI API docs.
2. Chain-of-Thought Prompting. Wei et al., 2022 (arXiv).
3. Self-Consistency. Wang et al., 2022 (arXiv).
4. Adaptive thinking. Claude API docs.
5. Pricing. Claude API docs.
6. Multi-agent coordination patterns. Anthropic.
7. Building agents with the Agent SDK. Anthropic.
8. Context Rot. Chroma research, 2025.
9. Demystifying evals for AI agents. Anthropic Engineering.
10. Lost in the Middle. Liu et al., 2023 (TACL).
11. Effective context engineering. Anthropic Engineering.
- V. AI Prompt Engineering: A Deep Dive. Anthropic panel (video).



1



2



3



4



5



6



7



8



9



10



11



V